Tests de performance

Avec K6 Grafana

https://grafana.com/docs/k6/latest/examples/tutorials/
https://github.com/imhah-hassan/k6-training

→ 24/03/2026

onepoint.
beyond the obvious

Agenda

Module	Date	Horaires	Salle
Module 6 Test de performance : Gatling	23-mars	13h45-17h00	Amphi sc.071, Bouygues

Logistique

- Assiduité et émargement
- Pauses
- Interactivité et questions
- Utilisation de laptops et smartphones
- Evacuation

Sommaire

0. Introduction aux tests de performance
1. Initialisation projet K6 (VSCode)
2. Développement Virtual User
3. Utilisation des données
4. Modèle de montée en charge
5. Cas pratique

0. Introduction aux tests de performance

Introduction

La gestion des risques au centre des tests de performance

Auditer la performance applicative

Capacité à identifier les points faibles et à obtenir un état des lieux du niveau actuel des performances

Identifier les Risques & les Exigences

Analyse technique et fonctionnelle des risques et des exigences de performance.

Formaliser les objectifs

Décrire les objectifs couvrants les Risques et les Exigences de performance.

Assurer la couverture

Associer les tests de performance aux objectifs pour assurer la couverture des risques et des exigences

Réaliser les tests de performance

Préparer et exécuter les tests. Analyser les résultats. Proposer des optimisations de la plateforme.

Amélioration continue

Analyser les incidents de performance et de disponibilité en production. Ajuster le modèle de conception des tests de performance

Quelques tristes exemples

Les objectifs de performance doivent être exprimés au **pic de fréquentation**/utilisation de l'application à tester.

Généralement les tests de performance permettent de s'assurer qu'en cas de pic, le service n'est pas dégradé.

09/08/2023

DÉCLARATION DE BIENS IMMOBILIERS: INDISPONIBLE EN PARTIE CET APRÈS-MIDI, LE SITE DES IMPÔTS REFONCTIONNE

14 mars 2023

Passe Navigo : la plateforme d'indemnisation de nouveau disponible après une perturbation

4 octobre 2023

Billets de train pour Noël : le site et l'application SNCF Connect saturés ce matin, retour à la normale

19/06/2023

LE SITE DE MÉTÉO-FRANCE TOMBE EN PANNE EN PLEINE SÉQUENCE ORAGEUSE

Vue d'ensemble : critères qualité logiciel ISO 25010

Qualité Logicielle
ISO - 25010

Fonctionnels	Performance	Compatibilité	Utilisabilité	Fiabilité	Sécurité	Maintenabilité	Portabilité
Exactitude Complétude Aptitude à l'usage	Temps de réponse Utilisation des ressources Capacité	Coexistence Interopérabilité	Intelligibilité Apprentissage Opérabilité Protection contre les erreurs Ergonomie Accessibilité	Maturité Disponibilité Robustesse Récupérabilité	Confidentialité Intégrité Rejet (non-rejet) Responsabilité Authenticité	Modularité Réutilisabilité Analysabilité Testabilité	Adaptabilité "Installabilité" Remplaçabilité
Couverture	Couverture	Couverture	Couverture	Couverture	Couverture	Couverture	Couverture
Tests Système Intégration Acceptation	Tests de performance	Tests d'intégration	Tests d'utilisabilité RGAA	PRA : Plan de reprise de l'activité Indicateurs niveaux de service (SLA) Tests de performance	Plan d'Assurance Sécurité Plan de réponse aux incidents de sécurité (PRI) Pentest et audit	Normes de développement/architecture Framework	Tests d'exploitabilité

Implémenter les tests de performance au sein de l'organisation

1. Stratégie de test de performance
- Basée sur la gestion des risques
- Orientée exigences métiers
- Définie l'organisation, les rôles et les responsabilités

2. Performance By Design
- Audit du code
- Revue par les paires
- Bonnes pratiques pour un code performant
- Contrôle automatique du code intégré à la gestion de version

3. Gérer un projet de performance
- Test de performance Agile dans le sprint
- Validation produit
- Sécurisation de la transition en production

4. Identifier les incidents de performance en production
- Définir les métriques de performance
- Monitorer les systèmes et les processus métier
- Analyse de logs
- Alertes et reporting

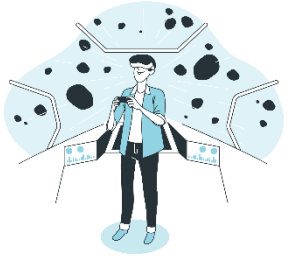
5. Mesurer et contrôler la performance et la disponibilité des applications
- Surveillance des parcours des utilisateurs réels
- Mesure de la disponibilité des processus métier
- Surveillance des applications et corrélation des évènements trans-systèmes.
- Alertes et reporting

Processus de réalisation d'un test de performance

Parcours utilisateur

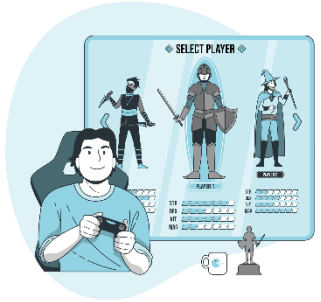
k6

Virtual User Script



Controller

k6



Load injector

Application Under Test

Monitoring

Conduire un projet de test de performance

1 PREPARER

- Analyse des risques
- Sélection des cas d'utilisation
- Définitions des objectifs de performance et des métriques associées
- **Identification des données**

2 RÉALISER

- Développement des scripts
- Implémentation des données dans les scripts et corrélations
- Préparation des données avec la volumétrie cible
- Définitions des modèles de montée en charge
- Mise en place de la plateforme d'injection de la charge
- Exécution des tests de validation globale (10% de la charge cible)

3 EXÉCUTER

- Alimentation des données
- Mise en œuvre des indicateurs de supervision
- Exécution des tirs de performance
- Analyse des résultats après chaque tir, débriefing et tuning

4 BILAN

- Analyse globale des exécutions et des bilans intermédiaires
- Rapport final avec conclusions et recommandation

Transfert de compétences

Types de test

Test de charge :

- Charge de travail supérieure à sa capacité normale.
- Simuler une situation de forte demande
- Capacité du système à gérer les pics et à maintenir un niveau de performance acceptable.

Test de stress :

- Conditions extrêmes, conditions aux limites
- Surcharge ou une saturation des ressources
- Capacité à résister à ces conditions sans tomber en panne ou réduire considérablement sa performance.

Test de montée en charge :

- Mesure la performance d'un système en augmentant progressivement la charge jusqu'à atteindre sa capacité maximale.

Test de endurance :

- Performance d'un système sur une période prolongée, généralement plusieurs heures ou jours
- Capacité à maintenir un niveau de performance acceptable sur une période de temps prolongée

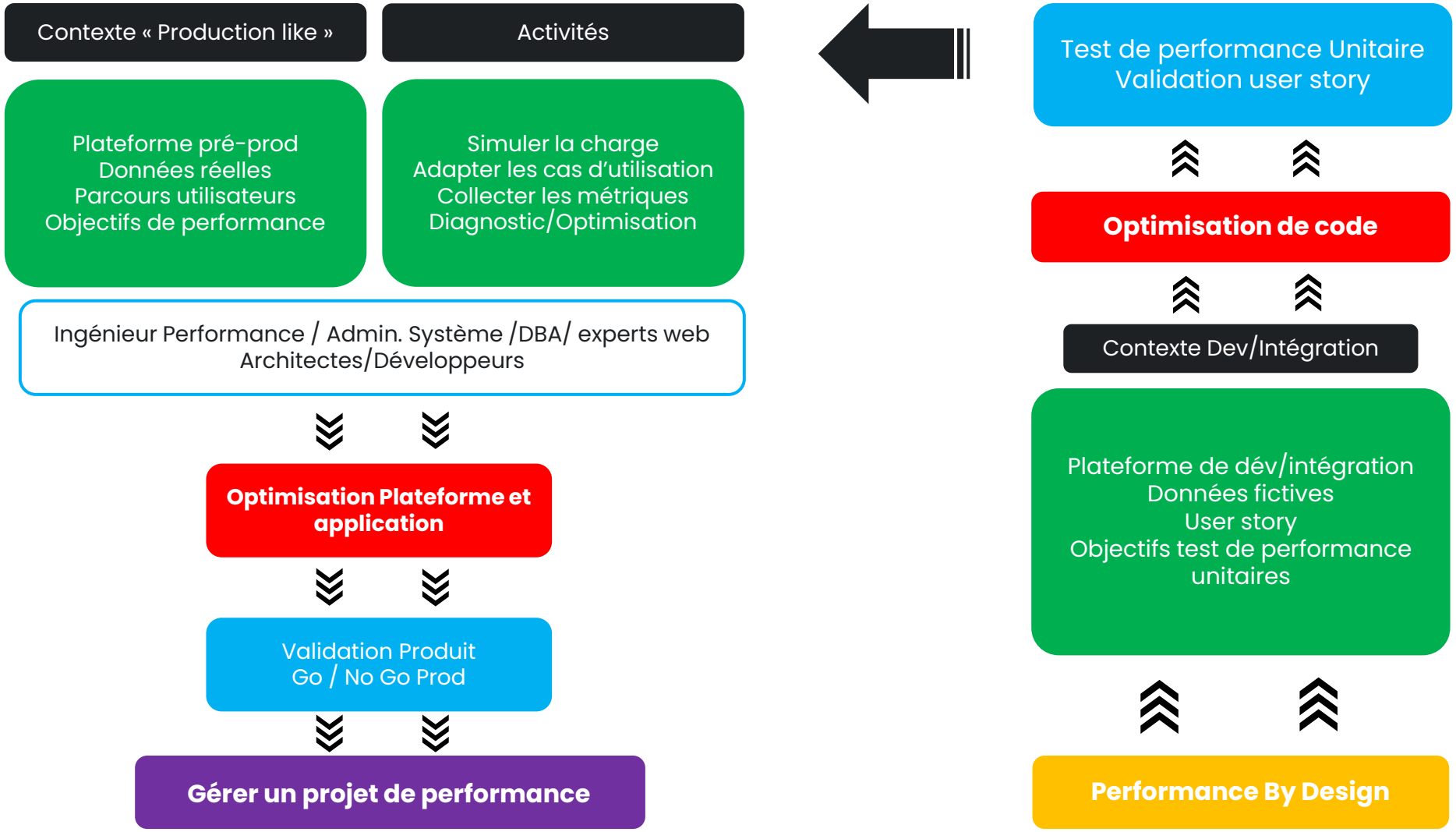
Test de configuration :

- Performance d'un système en modifiant sa configuration, telles que la mémoire, le processeur, le réseau, ou la base de données
- Impact de ces modifications sur la performance du système.

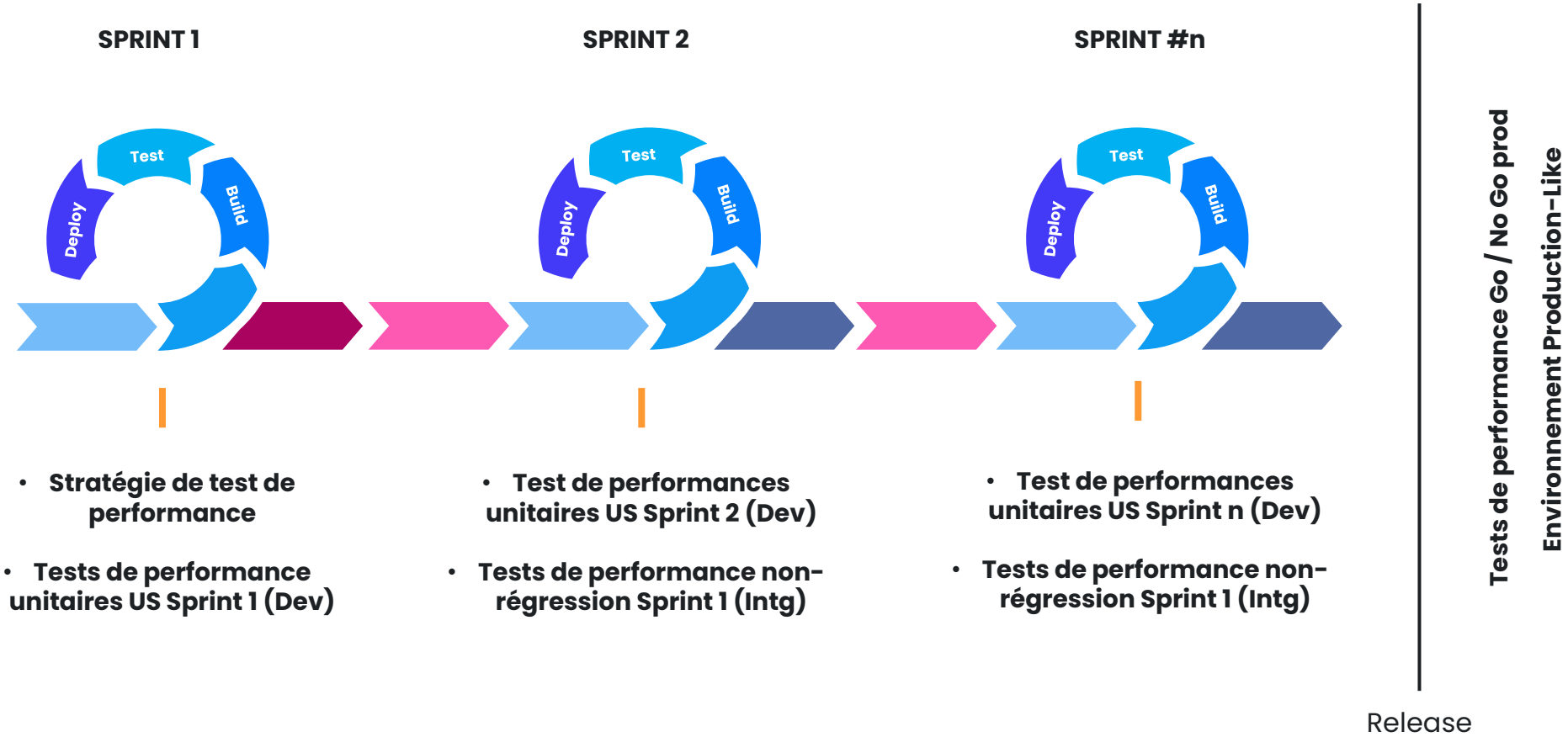
Test de volume :

- Performance d'un système avec une volumétrie de données importante.
- Capacité à maintenir un niveau de performance en cas d’augmentation du volume de données

Test de performance et agilité



Test dans le sprint



- Stratégie de test de performance
- Tests de performance unitaires US Sprint 1 (Dev)

- Test de performances unitaires US Sprint 2 (Dev)
- Tests de performance non-régression Sprint 1 (Intg)

- Test de performances unitaires US Sprint n (Dev)
- Tests de performance non-régression Sprint 1 (Intg)

Diagnostic des performances

- Métriques situation actuelle
- Temps de réponse
- Volumétrie de données
- Charge (transactions / s ..etc)

- Volumétrie de données
- Architecture cible
- Charge cible
- Temps de réponse attendus

- Construction du modèle de charge
- Injection de la charge
- Relevé des métriques
- Comparaison performances attendues / obtenues
- Analyse des écarts

- Recommandations tuning architecture
- Mise en place recommandations
- Injection de la charge
- Relevé des métriques
- Comparaison performances attendues / obtenues

1. Initialisation projet k6 (VSCode)

Présentation K6 Grafana

- k6 est écrit en Go, mais s'utilise en JavaScript. C'est l'outil favori des SRE et DevOps car il s'intègre parfaitement dans une CI/CD (GitLab, GitHub Actions).
- Contrairement à d'autres outils, k6 n'utilise pas un thread par utilisateur. Il utilise des "Virtual Users" (VU) légers gérés par le moteur Goja (JS runtime en Go).

2 modes Core vs Browser :

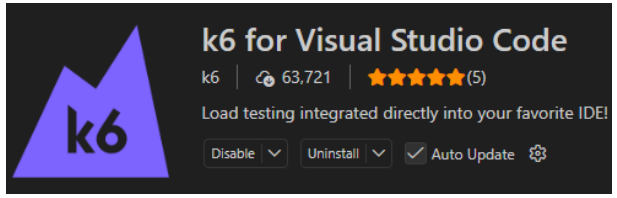
- Core (HTTP) : Simulation de requêtes réseau (très haute performance).
- Browser : Pilotage d'un vrai navigateur (Chromium) pour mesurer le rendu (LCP, FID).

Comparatif K6/ Gatling / NeoLoad / Jmeter

Caractéristique	Gatling	JMeter	NeoLoad	K6
Architecture	Basé sur Akka et Netty, modèle non-bloquant	Basé sur Java, utilise des threads Java	Architecture distribuée propriétaire	Basé sur Go, modèle non-bloquant
Script	Code Scala, approche DSL	Interface graphique + code XML/Java	Interface graphique + JavaScript	Code JavaScript, approche DSL
Évolutivité	Très bonne grâce au modèle d'acteurs	Limitée (1 thread par utilisateur virtuel)	Bonne avec distribution de charge	Excellente, très scalable (cloud natif)
Courbe d'apprentissage	Moyenne à élevée (nécessite connaissance de Scala)	Faible à moyenne	Faible à moyenne	Faible à moyenne (JavaScript)
Rapports	Rapports HTML natifs détaillés	Basiques, extensions nécessaires	Rapports riches et personnalisables	Rapports détaillés via Grafana / cloud
Licence	Open Source (Apache 2.0)	Open Source (Apache 2.0)	Commercial	Open Source (AGPL v3), Cloud commercial
Coût	Gratuit (version entreprise disponible)	Gratuit	Payant	Gratuit (open source) ou payant (Cloud Pro)
Interface	Code principalement, interface minimaliste	Interface graphique complète	Interface graphique riche	Interface en ligne de commande + dashboard Cloud
Intégration CI/CD	Excellente	Bonne	Bonne	Excellente, pensée pour CI/CD
Support	Communauté + support payant	Communauté	Support commercial	Communauté + support commercial
Maintenance des scripts	Facile (code structuré)	Plus complexe	Moyenne	Facile
Monitoring	Basique, intégrations possibles	Basique, plugins disponibles	Avancé, intégré	Avancé avec intégration Prometheus/Grafana
Données	Feeder puissant et flexible (CSV, JSON, JDBC, custom Scala)	CSV, base de données, plugins nécessaires pour formats avancés	Interface native de gestion des données, simplifiée	Gestion native des données via fichiers JSON/CSV, ou génération dynamique via JS
Variables de contexte	Excellente via Session, manipulation fluide avec Scala	Variables de contexte accessibles, parfois verbeux	Très ergonomique via interface visuelle	JavaScript, scénarios facilement dynamiques et paramétrables
Conception des scénarios	DSL Scala expressive et typée, très flexible pour enchaîner des scénarios complexes	Visuel ou XML, moins fluide pour les cas complexes	Design graphique ergonomique, bon pour scénarios métier	Gestion fine via JS, assertions puissantes et extensibles
Erreurs et assertions	Intégrée dans le DSL, assertions fines	Assertions de base, extensions possibles	Puissante via UI, assertions complexes faciles à paramétrer	Très flexible via code, scénarios de charge avancés
Paramétrage de la charge	Injection dynamique précise : ramp-up, palier, boucle infinie, etc.	Ramp-up configurable mais rigide comparé à Gatling	Très riche et accessible via interface	Paramétrage avancé via le code JS, scénarios de charge flexibles, mais nécessite scripting manuel.
Enregistrement de scénarios	Recorder HTTP disponible, enrichissement manuel nécessaire	Recorder HTTP intégré	Enregistreur avancé avec analyse automatique	Pas d'enregistreur natif, mais outils externes ou proxy
Debug et logs	Logs détaillés, débogage précis avec traces de session	Très verbeux, mais parfois fouillis	Interface riche et lisible pour debug	Console JS + dashboards intégrés, bonne lisibilité
Maintenance et évolutivité des scripts	Structure claire et modulaire, très maintenable	Scripts XML peu lisibles à grande échelle	Moyenne, dépend de la complexité des scénarios	Scripts JS modulaires, très bonne maintenabilité
Collaboration et versioning des scripts	Parfait pour Git, tout est du code propre	XML versionnable, mais conflits fréquents	Partage possible mais dépend du projet	Scripts JS faciles à versionner et collaborer
Extensions	Haute, intégration avec écosystème Scala/Java	Immense bibliothèque de plugins	API extensible, mais dépend du modèle commercial	Extensions JS via npm, très vaste écosystème
Tests multi-protocoles	HTTP, WebSocket, JMS	HTTP, FTP, JDBC, JMS, SOAP	Large support multi-protocole natif	HTTP, WebSocket, gRPC via extensions

Installation K6

- https://grafana.com/docs/k6/latest/set-up/install-k6/
- Environnements supportés : Linux, macOS, Windows, docker
- Possibilité d'utilisé l'IDE spécifique K6 Studio (https://grafana.com/docs/k6-studio/set-up/install/)
- Installer VSCode et NodeJS (version 20.0 et supérieur)
- Installer l'extension K6 for VSCode :
  - **npm init -y**
  - **npm i –g k6**
- IntelliSense (Autocomplétion) : **npm install --save-dev @types/k6**
- Vérification : exécuter k6




```
C:\github_himhah-op\k6-training>k6

        /\      Grafana   /‾‾/
   /\  /  \     |\  __   /  /
  /  \/    \    | |/ /  /   ‾‾\
 /          \   |   (  |  (‾)  |
/ ‾‾‾‾‾‾‾‾‾‾\   |_|\_\  \_____/

Usage:
  k6.exe [command]

Available Commands:
  archive     Create an archive
  cloud       Run a test on the cloud
```

k6 ne lit pas le TypeScript nativement. Vous devez utiliser un "bundler" (comme Webpack ou Rollup) pour transformer votre TS en JS avant de le lancer.

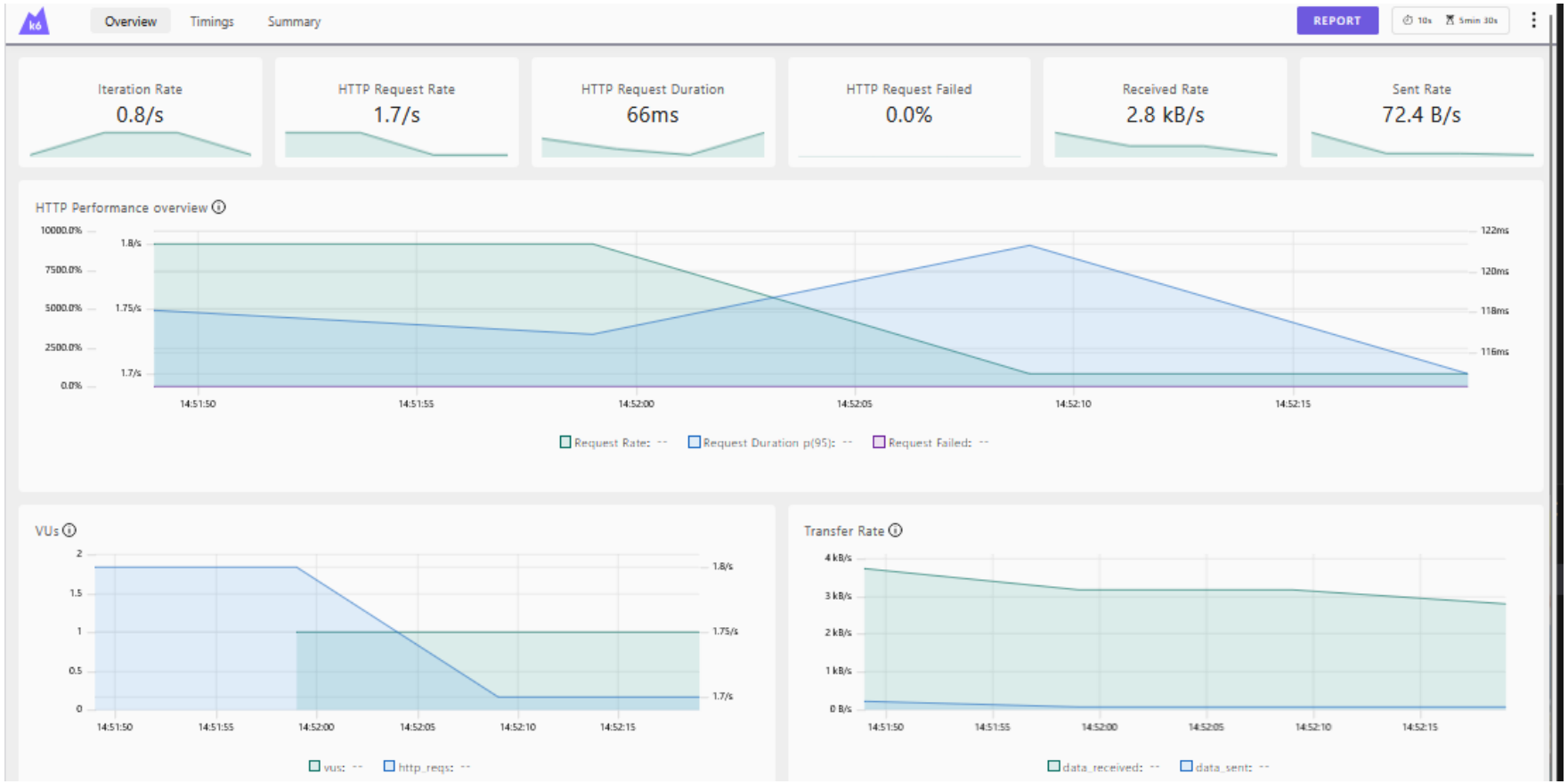
Exécution d'un test

- Créer le fichier jsconfig.json avec ce contenu
- Créer un test scripts\test.js avec ce contenu
- Lancer le test : k6 run scripts/test.js

```
        /\      Grafana   /‾‾/
   /\  /  \     |\  __   /  /
  /  \/    \    | |/ /  /   ‾‾\
 /          \   |   (  |  (‾)  |
/ ‾‾‾‾‾‾‾‾‾‾ \  |_|\_\  \_____/

     execution: local
        script: scripts/test.js
        output: -

     scenarios: (100.00%) 1 scenario, 1 max VUs, 10m30s max duration (incl. graceful stop):
              * default: 1 iterations for each of 1 VUs (maxDuration: 10m0s, gracefulStop: 30s)


 ╭─ TOTAL RESULTS ─────────────
```

Le rapport de test

- `SET K6_WEB_DASHBOARD=true`
- `k6 run –duration 5m scripts/test.js`
- Ouvrir `http://localhost:5665/ui/?endpoint=/`



Mode debug

Ne jamais debugger avec plusieurs VUs. **Toujours faire le debug avec 1 VU et 1 Itération.**

1. Utiliser console.log ou console.debug
2. Utiliser le mode debug http : `k6 run --http-debug scripts/test.js`

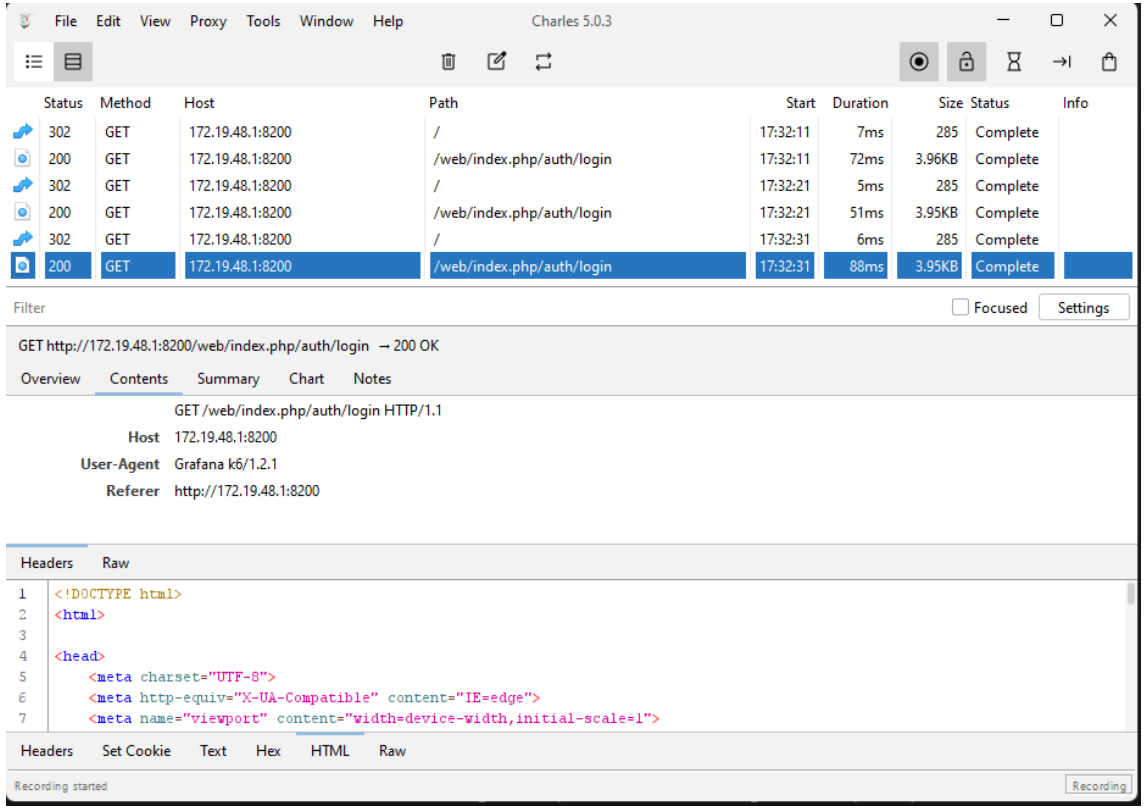
```
k6 run --vus 1 --iterations 1 --http-debug scripts/test.js
```

4. Utiliser un proxy http (Fiddler ou Charles proxy)

Démarrer Charles Proxy

Exécuter le test

```
SET HTTPS_PROXY=http://127.0.0.1:8888
k6 run --iterations 100 --insecure-skip-tls-verify scripts/test.js
```



Si le site est installé en localhost, y accéder avec l'IP du poste

La structure d'un test

INIT : Configuration du test

Tout ce qui est en dehors des fonctions export appartient à cette phase.

Importations de modules (http, check, sleep).

Options : "SLA" (Thresholds) et votre charge (VUs, durée).

Setup (optionnel)

S'exécute une seule fois, au tout début, avant que les VUs ne soient lancés.

Elle peut retourner des données (un jeton d'accès par exemple) qui seront transmises à chaque VU.

Fonction *default (obligatoire)*

Chaque VU l'exécute du début à la fin, puis recommence immédiatement (une "itération"), jusqu'à la fin du temps imparti.
Point de vigilance : Ne déclarez jamais de variables lourdes ou ne lisez pas de fichiers *à l'intérieur* de cette fonction, sinon vous allez ralentir vos mesures de performance inutilement.

teardown (optionnel)

S'exécute une seule fois à la fin du test, même si le test a échoué.
Pratique pour laisser l'environnement propre après un test.

```
import http from 'k6/http';
import { sleep, check } from 'k6';

// 1. INIT : Configuration du test
export const options = {
  vus: 5,
  duration: '10s',
  thresholds: {
    http_req_duration: ['p(95)<500'], // 95% des requêtes doivent être sous 500ms
  },
};

// 2. SETUP : On peut récupérer un token global ici si besoin
export function setup() {
  console.log('--- Démarrage du test ---');
  return { server: 'https://opensource-demo.orangehrmlive.com' };
}

// 3. VU CONTEXT : La boucle principale
export default function (data) {
  const res = http.get(`${data.server}/web/index.php/auth/login`);

  check(res, {
    'page login accessible': (r) => r.status === 200,
    'présence du formulaire': (r) => r.body.includes('login'),
  });

  sleep(1); // "Think time" pour simuler un humain
}

// 4. TEARDOWN : Nettoyage final
export function teardown(data) {
  console.log('--- Test terminé avec succès ---');
}
```

SharedArray

Le SharedArray est charge les données dans la mémoire de manière partagée (si vous avez 1000 VU le fichier JSON n'est chargé qu'une seule fois au lieu de 1000).

Etapes pour l'implémentatio

1. Préparer le fichier de données (data.json)

2. Implémentation du SharedArray dans le script

```
import { SharedArray } from 'k6/data';
const accountData = new SharedArray('comptes OrangeHRM', function () {
   // open() lit le fichier, JSON.parse() le transforme en objet JS
   const data = JSON.parse(open('./data/users.json'));
return data; });
```

3. Sélection de la donnée

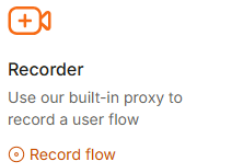
```
// On utilise le modulo (%) pour boucler sur le fichier si on a plus d'itérations que de comptes const currentIndex = __ITER %
accountData.length; const currentUser = accountData[currentIndex];
```

4. Utilisation dans la requête

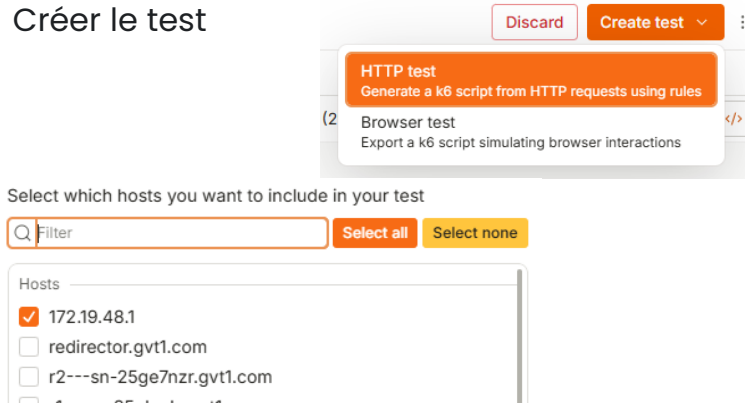
```
const res = http.post(`https://opensource-demo.orangehrmlive.com/web/index.php/auth/validate`,
             { _token: `CSRF_TOKEN_ICI`, // À corréler comme vu précédemment
             username: `${currentUser.user}`, password: `${currentUser.pass}`,
});
```

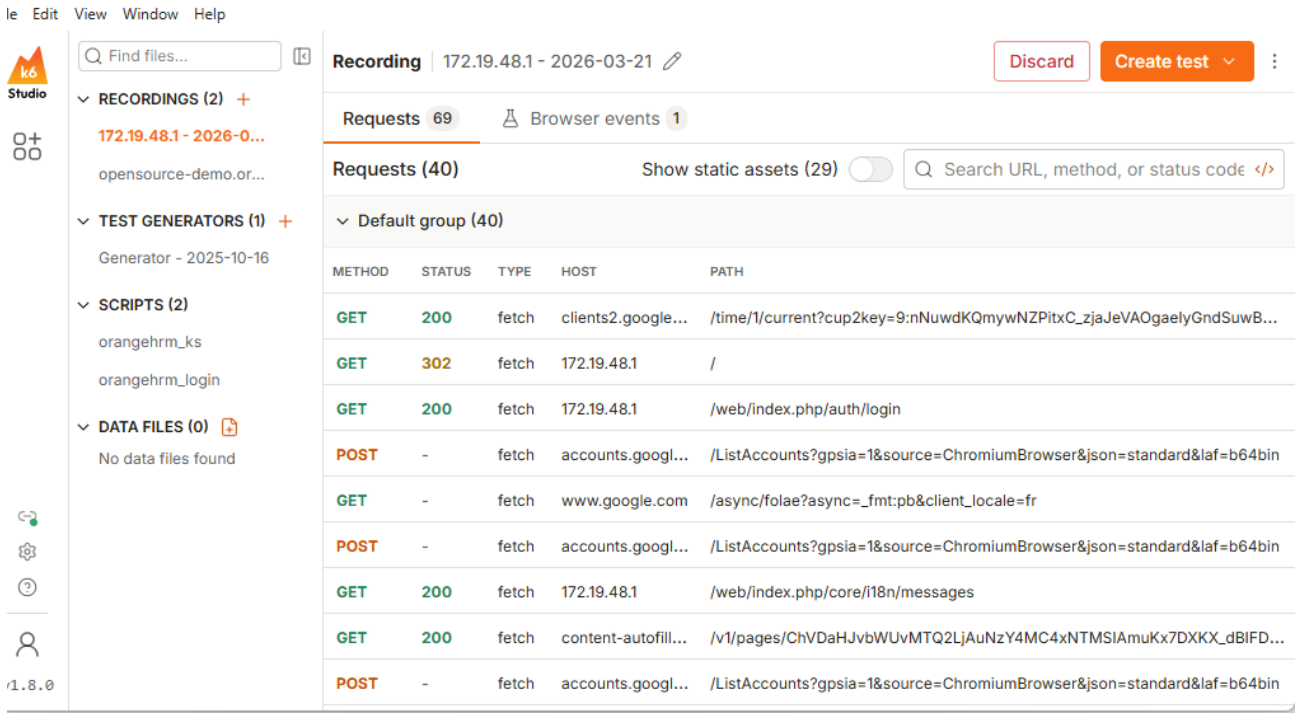
Enregister un virtual user : K6 studio

- ***Installer K6 Studio : https://github.com/grafana/k6-studio/releases***
- Lancer l'enregistrement



- Enregistrer le login/logout
- Créer le test





Enregister un virtual user : har converter

https://grafana.com/docs/k6/latest/using-k6/test-authoring/create-tests-from-recordings/using-the-har-converter/

https://github.com/grafana/har-to-k6

- Installer har converter

```
npm install -g har-to-k6
```

- Enregistrer le har (dev tools chrome/edge/firfox … etc)

```
har-to-k6 Tools/orangehrm.har -o scripts/employee-har.js
```

Remarques :

- K6 Studio génère des scripts plus concis et plus simple à adapter.
- Pour des sites https, la configuration du certificat dans K6 Studio est plus complexe.

Exercice 1 : Enregistrement et rejeu

Ne jamais lancer un test de charge sur un site sans l'accord du propriétaire

0.1. Lancer ***K6 Studio*** et démarrer l'enregistrement

0.2. Réaliser le test manuel sur orangehrm (http://xxxxxx)

Login → Création Employé → Saisie du détail → Recherche Emplyé par nom → Modification des coordonnées → Recherche Employé par matricule → Suppression de l'employé → Logout

0.4. Vérifier que les requêtes sont bien enregistrées.

0.5. Analyser le code généré et isoler le login et le logout.

0.6. Construire un scenario uniquement avec le login et le logout.

0.7. Exécuter le test (avec un seul utilisateur et sans supprimer les temps d'attente) et analyser les résultats et corriger les erreurs

Si l'application est en localhost, sur firefox positionner le paramètre *network.proxy.allow_hijacking_localhost* a ***true (about:config)***
https://learn.microsoft.com/en-us/microsoft-cloud/dev/dev-proxy/how-to/intercept-localhost-requests?pivots=client-operating-system-windows

6. Développement Virtual User

Processus de développement

L'analyse préalable permet :

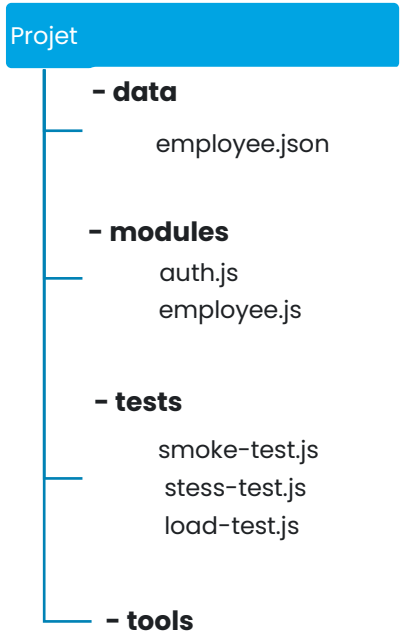
- la définition des objectifs (ex API de paiement 500 transactions/seconde temps de réponse < 200ms)
- l'identification des scénarios critiques (ex parcours d'achat complet, de la connexion à la confirmation de commande)
- La définition du modèle de charge : (Ex : pics attendus à 2000 utilisateurs simultanés pendant les soldes, objectif de temps de réponse moyen inférieur à 500ms)

Enregistrement	Organisation du code	Corrélations / Assertions	Variabilisations	Modèle de charge	Exécution et Monitoring
Gatling Recorder : mvnw gatling:recorder Configuration du proxy localhost:8000 Capture du trafic HAR Enregistrer une session complète de navigation	Organisation des ChainBuilder. Séparation du code des ChainBuilder et des scéanrios. Identification des scénarios réutilisables. Identification des données nécessaires Développement des utilitaires (pause aléatoire, calcul sur les dates … etc)	Charger le har capturer dans Firefox pour identifier les corrélations. Identification des tokens : Extraire les jetons CSRF, sessionID, etc. Identification des données techniques échangées entre le client et le serveur. Adentification dans les requêtes, les headers, les body (dans les fichiers ressources xxx_request.json) Vérification des données capturées (${sessionId} est correctement extraite) Vérification fonctionnelle	Création de feeders Injection des données Développements spécifiques pour générer ou extraire les données nécessaires	Définir les profils d'injection. Construire des scénarios réalistes. Valider le modèle de charge et le reporting sur un tir à faible volume.	Industrialisation le déploiement des scripts, l'exécution du test et la génération du rapport. Vérifier le retour à l'état initial après chaque tir. Mettre en place le monitoring du tir (influxDb, grafana) Vérifier la mise en place du monitoring de la cible.

Validation des scripts

Organisation du code et processus de mise au point des tests

1. Capture (K6 studio ou har converter) et suppression du "bruit« (supprimez les appels vers les domaines tiers (Google Fonts, Analytics) et les fichiers statiques (.png, .css))
2. Nettoyage de code
3. Adopter une structure modulaire
4. Implémentation de la Logique Métier : corrélation et paramétrage
5. Conception des scénarios : séparez la modélisation de la charge du code. Utilisez des scenarios pour exécuter plusieurs parcours en même temps.
6. Validation des scripts : debug et run
7. Lancement test de charge à faible charge
8. Tirs de charge cibles



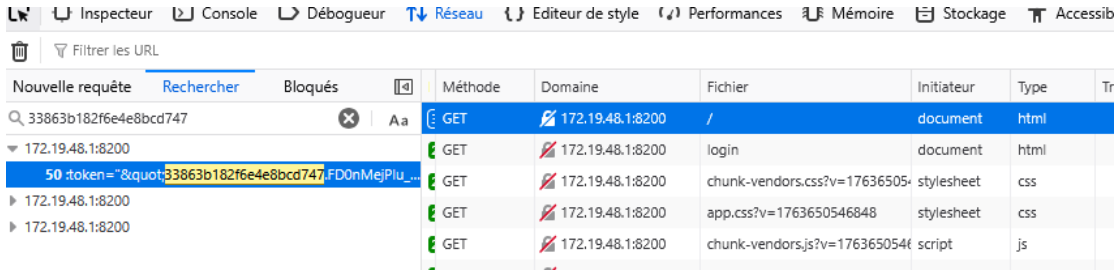
Corrélation et assertions

La corrélation consiste à capturer une valeur dynamique envoyée par le serveur pour la renvoyer dans les requêtes suivantes.

Exemple dans la requête /auth/validate, le token a certainement était envoyé dans une requête précédente

```
url = http.url`http://172.19.48.1:8200/web/index.php/auth/validate`;
resp = http.request(
      "POST",
      url,
      JSON.parse(
        `{"_token":"b73b5e7f9a.SYuKJCRyc-fxxbabYUOWJoPbqHspOgAGwCPhdFX5yVA.Dv_HHUFDCqWkmoXxDQ34TNqN-RlPWWx-q3OzOAPLkAkF7_J-
SjYnrpigxQ","username":"Admin","password":"Orangehrm$2026"}`,),
      params,
    );
```

Utiliser les dev tools du browser pour rechecherche la requête qui a envoyée ce token




```
<div id="app">
<auth-login
        :token="&quot;33863b182f6e4e8bcd747.FD0nMejPlu_dAA8az281eXoMol8Ah5J6XQ2aqiFrM60.
        :login-logo-src="&quot;\/web\/images\/ohrm_logo.png&quot;"
        :login-banner-src="&quot;\/web\/images\/ohrm_branding.png?v=1763650546848&quot;"
        :show-social-media="true"
        :is-demo-mode="false"
        :authenticators="[]"
```

Corrélation et assertions

Utiliser la syntaxe check pour valider le retour puis l'objet response pour lire le token

```
check(response, {
'01-status 200': (r) => r.status === 200,
});
const csrfMatch = response.body.match(/:token="&quot;(.*)&quot;/);
const token = csrfMatch ? csrfMatch[1] : null;
console.log ("Token   ", token);
```

Exemple de check

Type de validation	Propriété k6	Exemple court	Explication
Statut HTTP	status	(r) => r.status === 200	Vérifie que l'appel s'est bien passé (Code 200, 201, 204...).
Contenu Texte	body	(r) => r.body.includes('Welcome')	Cherche une chaîne de caractères dans le HTML ou texte brut.
Donnée JSON	json()	(r) => r.json('id') === 42	Extrait et valide une valeur précise d'une API JSON.
Taille du Body	body.length	(r) => r.body.length > 0	Évite de valider des réponses vides ou des pages blanches.
Headers HTTP	headers	(r) => r.headers['Content-Type'].includes('json')	Valide le format de la réponse (ex: s'assurer que c'est du JSON).
Performance locale	timings.duration	(r) => r.timings.duration < 500	Alerte si cette requête précise prend plus de 500 ms.

Si vous voulez qu'un échec de validation **bloque** réellement le test (par exemple, si la première requête de login échoue, inutile de continuer le reste du scénario), vous devez coupler votre check avec un fail de k6 :

```
const checkOk = check(res, {
            'statut est 200': (r) => r.status === 200,
});
if (!checkOk) {
            fail('Erreur critique : impossible de joindre le site !');
}
```

Exemple d’extraction de données

Technique	Syntaxe / Sélecteur	Exemple court k6	Explication
JSON (Accès direct)	res.json('chemin.cle')	const id = res.json('user.id');	k6 extrait directement une propriété d'un body JSON (très rapide).
JSON (JS Standard)	res.json().propriete	const name = res.json().data[0].name;	On parse le JSON et on navigue dedans avec du JavaScript classique.
Sélecteurs CSS (HTML)	res.html().find('selecteur')	const token = res.html().find('input[name="id"]').attr('value');	Cherche un élément HTML par sa classe, son ID ou son attribut (remplace XPath).
RegEx (Texte brut)	/motif/ ou new RegExp()	const match = res.body.match(/id="([^"]+)"/); const id = match[1];	Recherche par motif textuel. Parfait quand la réponse n'est ni du JSON propre, ni du HTML.

Exercice 2 : Corrélations

0.1. Séparer les différentes actions du code.

0.2. Utiliser les fonctions pour créer les parcours suivants :

Login ➔ Consultation employé ➔ Logout

Login ➔ Ajout employé ➔ Details ➔ Contact ➔ Logout

Login ➔ Suppression employé ➔ Logout

0.3. Identifier les données qui nécessitent une corrélation en utilisant le har enregistré.

0.4. Ajouter les « check » et extraire les données et paramétrer les requêtes.

0.5. Exécuter le test au fur à mesure et ajouter les debugs des variables extraites.

0.6. Séparer le code des actions dans un module dédiée.

0.7. Exécuter le test (avec un seul utilisateur et avec des pauses aléatoires) et analyser les résultats et corriger les erreurs.

3. Utilisation des données

SharedArray

1. Generation des données avec faker

```
# Generate json fake data
npm install --save-dev @faker-js/faker
npm install --save-dev @types/faker
cd data
node generate-data.js
```

2. Définition du SharedArray

```
const accountData = new SharedArray('employees', function () {
    // open() lit le fichier, JSON.parse() le transforme en objet JS
    const data = JSON.parse(open('../data/employees.json'));
    return data;
});
```

(26) Records Found

Id	First (& Middle) Name	Last Name	Job Title	Employment Status	Sub Unit	Supervisor	Actions
8606025	Buddy	Nolan					
3349038	Carrie	Ruecker					
2611107	Charlene	Dach					
1261411	Cindy	Veum					
0661826	Doyle	Doyle					
6101056	Drew	Hayes					
1122712	Fred	Will					

3. Sélection de la donnée

```
// On utilise le modulo (%) pour boucler sur le fichier si on a plus d'itérations que de comptes
const currentIndex = __ITER % accountData.length;
const employee = accountData[currentIndex];
```

4. Lancer le test et vérifier que les employés sont bien ajoutés dans l'application

```
k6 run tests\smoke-test.js
```

CSVFeeder

CSVFeeder est un un “CSV queue service” : les datasets CSV sont chargés dans des ***Collections*** puis lus/consommés via des endpoints orientés test de charge.
CSVFeeder est une API REST pour charger des fichiers CSV dans Redis et les consommer comme des files de données (collections).
Une collection = un nom + des en-têtes + une liste de lignes.
Les lignes sont stockées en liste Redis, ce qui permet des opérations de type FIFO/rotation très rapides.

- API principales (hors gestion utilisateurs/projets) :
- GET /collections : liste les collections disponibles.
- POST /:collection/upload : importe un CSV dans une collection (file, separator/delimiter, overwrite).
- GET /:collection/count : renvoie le nombre de lignes.
- GET /:collection/line/:index : lit une ligne par index.
- GET /:collection/random : lit une ligne aléatoire.
- GET /:collection/rows?page=1 : lit les lignes paginées (100 par page).
- POST /:collection/add : ajoute une ligne en fin de collection.
- POST /:collection/pop : lit et supprime la première ligne (FIFO).
- POST /:collection/rotate : déplace la première ligne en fin de liste et la renvoie.
- DELETE /:collection : supprime entièrement une collection.

https://github.com/imhah-hassan/CSVFeeder

Exercice 3 : ShareArray

0.1. Reprendre l’exercice 2 et remplacer les données de l’employé (nom, prénom, matricule) par un fichier csv

0.2. Le test de charge comporte 3 parcours différents :

Login ➔ Consultation employé ➔ Logout

Login ➔ Ajout employé ➔ Details ➔ Contact ➔ Logout

Login ➔ Suppression employé ➔ Logout

0.3 Proposer une méthode pour gérer les donnés des différents parcours

6. Modèle de montée en charge

Répartition de la charge

Type de test	Objectif	Configuration typique
Smoke test	Vérifier que le script fonctionne	1 VU, 1 min
Load test	Valider sous charge nominale	10–50 VU, 5–30 min
Stress test	Trouver la limite de rupture	VU progressifs jusqu'à dégradation
Soak test	Détecter les fuites mémoire	Charge nominale, 1–8 heures

Smoke test

```
export const options = {
  vus: 1, // 1 seul utilisateur
  duration: '30s', // Très court
  thresholds: { http_req_duration: ['p(99)<1500’]
};
```

Stress test

```
export const options = {
  stages: [
    { duration: '2m', target: 100 },
    { duration: '5m', target: 100 },
    { duration: '2m', target: 200 },
    { duration: '5m', target: 200 },
    { duration: '2m', target: 300 },
    { duration: '5m', target: 300 },
    { duration: '2m', target: 0 },
  ],
};
```

Load test

```
export const options = {
  stages: [
   { duration: '2m', target: 50 },
   { duration: '5m', target: 50 },
   { duration: '2m', target: 0 }],
  thresholds: {
     http_req_duration: ['p(95)<500’],
     http_req_failed: ['rate<0.01’]}
};
```

Soak test

```
export const options = {
  stages: [
    { duration: '2m', target: 40 },
    { duration: '8h', target: 40 },
    { duration: '2m', target: 0 },
  ],
};
```

Exercice 4 : Modèle de charge

0.1. Reprendre l'exercice 3

0.2. Ajouter la répartition de charge suivante :

- 80% Login ➔ Consultation employé ➔ Logout
- 10% Login ➔ Ajout employé ➔ Details ➔ Contact ➔ Logout
- 5% Login ➔ Suppression employé ➔ Logout

0.3 Définir un modèle de charge ouvert : montée en charge de 2 VU toutes les 30 secondes jusqu'à 10 VU. Maintenir la charge pendant 10 mn

0.4 Pour lancer ce tir il est nécessaire de déployer l'application OrangeHRM localement (ou sur un serveur dédié)

6. Monitoring temps réél

Monitoring InfluxDb/Grafana

https://grafana.com/docs/k6/latest/results-output/real-time/influxdb/

K6 peut envoyer ses métriques en temps réel vers InfluxDB (v1.8 est la plus simple à configurer avec k6) via le flag --out. Grafana lit ensuite InfluxDB pour afficher un dashboard live.

```
# Lancer K6 avec export vers InfluxDB
k6 run tests/load-test.js --out influxdb=http://localhost:8086/orange_metrics
```

Configuration de la source de données

1. Ouvrez votre navigateur sur http://localhost:3000 (Login par défaut : admin / admin).
2. Allez dans Connections > Data sources et cliquez sur Add data source.
3. Choisissez InfluxDB.
4. Configurez l'URL : http://localhost:8086 (ou l'IP de votre machine si vous êtes sur Docker-Mac/Windows).
5. Dans la section Database (en bas), écrivez : k6.
6. Cliquez sur Save & Test. Vous devriez voir un message vert "Data source is working".

Configuration du dashboard

1. Dans Grafana, allez dans Dashboards > New > Import.
2. Dans le champ "Import via grafana.com", entrez l'ID officiel : 2587 (ou 10660 pour une version plus moderne).
3. Cliquez sur Load.
4. Sélectionnez votre source de données InfluxDB créée à l'étape 3.
5. Cliquez sur Import.

Analyse des résultats

Indicateur	Définition	Utilité en Performance
Moyenne (Mean)	Somme de toutes les valeurs divisées par le nombre de requêtes.	**A éviter**. Elle lisse les extrêmes. Si 1 utilisateur attend 10s et 9 attendent 1s, la moyenne est de 1.9s (personne n'a vécu 1.9s).
Percentile (p90, p95, p99)	Point sous lequel se trouve X% des requêtes.	**Recommandé**. Le $p(95) = 500ms$ signifie que 95% de vos utilisateurs ont attendu *moins* de 500ms (et 5% ont attendu plus). C'est l'expérience utilisateur réelle.
Écart-type (Std Dev)	Mesure de la dispersion des données autour de la moyenne.	Mesure la **stabilité**. Un écart-type faible = système prévisible. Un écart-type élevé = système instable (micro-coupures, saturation ponctuelle).

Erreurs & Débit

Avant de regarder les temps de réponse, vérifiez que le test est valide.

Taux d'erreurs (Error Rate) :

S'il y a plus de 1% d'erreurs (HTTP 500, timeout), l'analyse des temps de réponse est faussée (une erreur 500 répond souvent beaucoup plus vite qu'une page normale !).

Débit (RPS / Throughput) :

Est-ce que le serveur a traité la charge demandée ? Si vous vouliez 100 req/s et que le serveur plafonne à 50 req/s, le système sature.

Moyenne et Médiane

Si $Moyenne \approx p50$: Vos temps de réponse sont homogènes (courbe stable).

Si $Moyenne >> p50$: Vous avez des "valeurs aberrantes" (outliers). Quelques requêtes très lentes tirent la moyenne vers le haut.

Écart-type

Comparez l'écart-type à la moyenne.

Règle empirique :

Si l'Écart-type est supérieur à la moitié de la Moyenne ($SD > \frac{\bar{x}}{2}$), le système est considéré comme instable.

Les utilisateurs ne vivent pas tous la même expérience.

Percentiles

On confronte les percentiles aux objectifs fixés par le métier (les SLAs).

Exemple d'objectif : $p(95) < 500ms$ et $Max < 2s$.

Si le $p(95)$ passe, mais que le $p(99)$ s'envole à 10 secondes, cela signifie que 1% de vos utilisateurs vit une expérience exécrable.

Sur 100 000 requêtes, cela fait 1 000 utilisateurs frustrés.

Infrastructure

On cherche le pourquoi :

Dégradation linéaire :

Les percentiles augmentent en même temps que les VUs ➔ Le code manque d'efficacité (algorithme, requêtes DB lourdes).

Dégradation brutale (Effet falaise) :

Les percentiles sont plats puis explosent d'un coup ➔ Verrou de base de données (deadlock), saturation de la bande passante, ou épuisement du pool de connexions HTTP/DB.

6. Cas pratique
Gestion de projets Agile avec Taiga.io

Cas pratique

Installer taiga.io sur docker (en local ou sur un serveur commun) :

```
git clone https://github.com/taigaio/taiga-docker.git
cd taiga-docker
git checkout stable
./launch-taiga.sh
/taiga-manage.sh createsuperuser
```

1. Créer un projet Scrum (vérifier que le module Issue est actif : Settings → Project → Modules
2. Avec Gatling capturer le parcours : Login → Issues List → Create Issue (Subject / Description / Type / Severity / Priority) → Update Issue (Status, Priority) → Search Issue → Delete Issue → Logout
3. Définir 4 parcours :
   a. Login → Issues List → Search Issue → Logout
   b. Login → Issues List → Issues List → Create Issue (Subject / Description / Type / Severity / Priority) → Logout
   c. Login → Issues List → → Update Issue (Status, Priority) → Logout
   d. Login → Issues List → Search Issue → Delete Issue → Logout
4. Répartir la charge comme suit : a : 75% - b : 15% - c : 10% - d : 10%
5. Réaliser les tirs de performance et identifier les limites de performance de l'application.

Merci

→

onepoint.
beyond the obvious